

Informe Técnico: Lotería Descentralizada basada en Smart Contracts

Proyecto Final de Solidity e Interacción ERC-20 / ERC-721

Autor:

Milton Alejandro Cuervo Ramirez

Ingeniería de Sistemas
Universidad de Antioquia

26 de mayo de 2026

1. Descripción General y Flujo del Sistema

Este proyecto académico nace con el propósito de diseñar un entorno de lotería digital y automatizada sobre la blockchain de Ethereum. El sistema ha sido programado de manera íntegra en un único archivo fuente para facilitar su compilación e interacción directa en Remix IDE. A diferencia de un sistema tradicional centralizado, esta solución distribuye la confianza entre contratos inteligentes que operan de forma autónoma. El juego se apoya en una economía cerrada donde los usuarios utilizan Ether para adquirir un token ERC-20 de utilidad, el cual posteriormente intercambian por boletos de lotería únicos representados bajo el estándar de NFT ERC-721.

El flujo de operaciones comienza cuando un usuario envía Ether al contrato principal llamando a la función de compra de tokens. Para garantizar que la experiencia de usuario sea fluida y no requiera de registros manuales, el contrato principal evalúa si la dirección del remitente es nueva. De ser así, se dispara un proceso interno que despliega de manera dinámica un contrato auxiliar exclusivo para ese usuario. Una vez que el usuario posee tokens, puede cambiarlos por boletos de lotería. El sistema descuenta los tokens, incrementa un contador global que actúa como identificador del boleto y acuña un NFT representativo directamente a la billetera del comprador, registrando este mismo identificador en su respectivo contrato auxiliar. El ciclo concluye cuando el administrador del contrato decide ejecutar el sorteo, momento en el cual se selecciona un boleto al azar, se identifica al propietario, se cobra una pequeña comisión administrativa y se le transfiere el pozo restante de Ether.

2. Arquitectura del Software y Lógica del Código

Desde la perspectiva de la ingeniería de software, se optó por una arquitectura descentralizada de tres capas que interactúan de forma estrecha. El corazón del sistema es el contrato principal, **DecenLottery**, el cual hereda de las librerías **ERC20** y **Ownable** de **OpenZeppelin**. Este contrato no solo actúa como el token de utilidad de la plataforma, sino que asume el rol de “fábrica” de contratos (*factory pattern*) al ser el responsable de inicializar y desplegar los contratos individuales **UserContract**. Al delegar el registro de los boletos a estos contratos auxiliares individuales en lugar de mantener arreglos dinámicos gigantescos dentro del contrato principal, logramos optimizar el consumo de gas de almacenamiento en la blockchain, una buena práctica clave en el desarrollo Web3.

Por otro lado, el contrato **LotteryNFT** hereda de **ERC721** y tiene como propietario inicial al contrato principal de la lotería. Esto significa que únicamente el contrato principal cuenta con los permisos necesarios para acuñar boletos, protegiendo el sistema contra emisiones maliciosas o fraudulentas de NFTs gratuitos. En cuanto a las funciones de interacción, la lógica de compra de tokens utiliza conversiones matemáticas a nivel de Wei (la unidad más pequeña de Ether) para manejar los 18 decimales del estándar ERC-20 sin perder precisión aritmética. El sistema también cuenta con simetría económica, permitiendo que cualquier usuario devuelva sus tokens sobrantes mediante la función de devolución y reciba de vuelta su Ether correspondiente de manera justa.

Durante el proceso de compra de boletos, la función correspondiente valida que el balance del usuario sea el adecuado, transfiere los tokens directamente a las reservas del contrato de lotería mediante llamadas internas y utiliza un bucle para procesar la cantidad de boletos solicitada. En cada iteración del bucle, el contrato incrementa un contador secuencial para evitar colisiones de identificadores y realiza dos operaciones de escritura cruzada: registra el boleto en el contrato auxiliar del usuario y ordena al contrato ERC-721 el minto del NFT.

3. Análisis de Seguridad y Buenas Prácticas

La seguridad ha sido un pilar fundamental en el diseño de este proyecto. El principal vector de ataque en contratos que gestionan y envían fondos es la reentrada (*reentrancy*). Para mitigar este riesgo, el código implementa estrictamente el patrón *Checks-Effects-Interactions* (Verificaciones-Efectos-Interacciones). Esto significa que el contrato actualiza todos sus estados locales y saldos de tokens internos antes de interactuar con direcciones externas para enviar Ether. Si un contrato malicioso intentara realizar una llamada recursiva durante el reembolso de Ether, la función fallará inmediatamente porque el balance interno del usuario ya habrá sido debitado a cero.

Adicionalmente, se implementaron controles de acceso mediante el modificador `onlyOwner` en las funciones críticas, asegurando que el sorteo del ganador solo pueda ser iniciado por el administrador de la plataforma. La lógica de generación del ganador utiliza una fórmula de pseudo-aleatoriedad mediante el hashing criptográfico de variables del bloque actual de Ethereum, como la marca de tiempo y el valor aleatorio del validador. En la defensa de este proyecto, es oportuno aclarar que este método es viable para fines académicos, pero para redes principales se debe migrar a soluciones descentralizadas como Chainlink VRF, dado que las variables de los bloques son predecibles y manipulables por los validadores si el pozo del premio fuese muy elevado.

4. Registro de Pruebas y Evidencias en Remix IDE

El comportamiento del sistema fue validado de manera exhaustiva en el entorno de pruebas local Remix VM. El flujo de pruebas ejecutado y registrado en las capturas adjuntas demuestra la correcta integración de todos los componentes:

4.1. Fase de Despliegue e Interacción ERC-20

En esta primera fase de pruebas, se compiló el archivo unificado sin generar advertencias ni errores en el compilador de Solidity. El contrato principal se desplegó utilizando la cuenta administradora. Posteriormente, se verificó que el balance inicial de la reserva de tokens en el contrato fuera el correcto (10,000 unidades completas).

Cambiando a una cuenta de pruebas de usuario, se ejecutó la compra de tokens enviando una cantidad de Ether superior a la necesaria para comprobar que el mecanismo de reembolso del excedente funcionaba adecuadamente. Asimismo, se consultó el balance del usuario para confirmar la recepción de los tokens y se verificó que el contrato auxiliar se creara de forma automática en su registro global. Para cerrar el ciclo económico, el usuario devolvió una porción de sus tokens y el contrato le reintegró el Ether de forma inmediata.

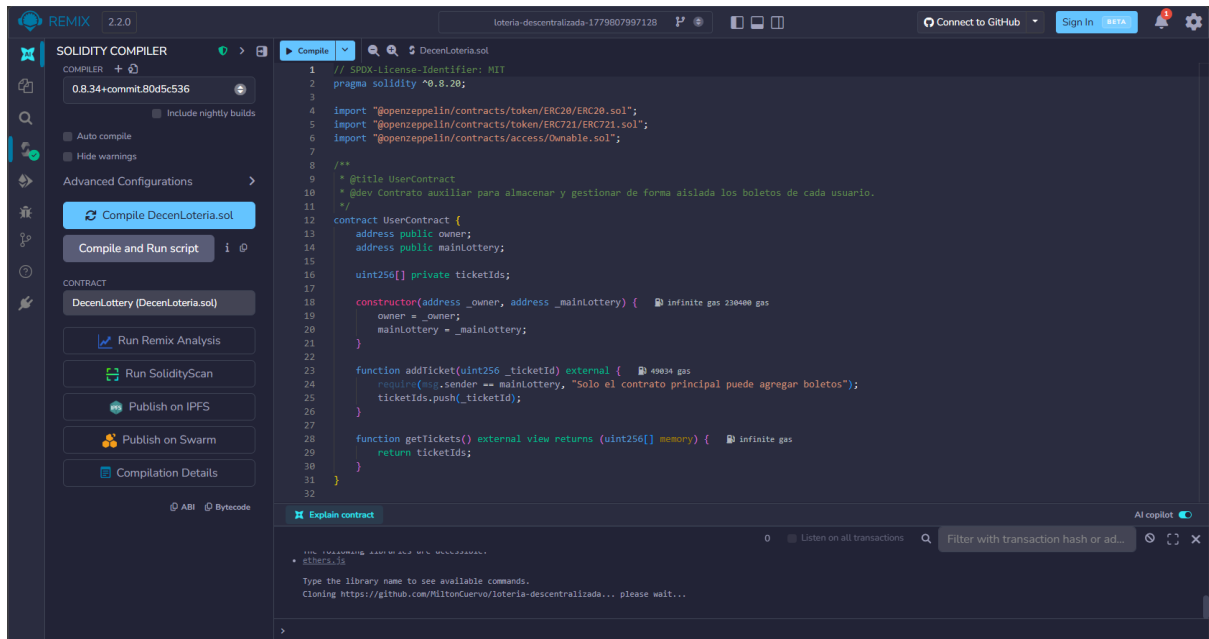


Figura 1: Compilación exitosa del archivo unificado de Solidity en Remix IDE utilizando el compilador versión 0.8.34.

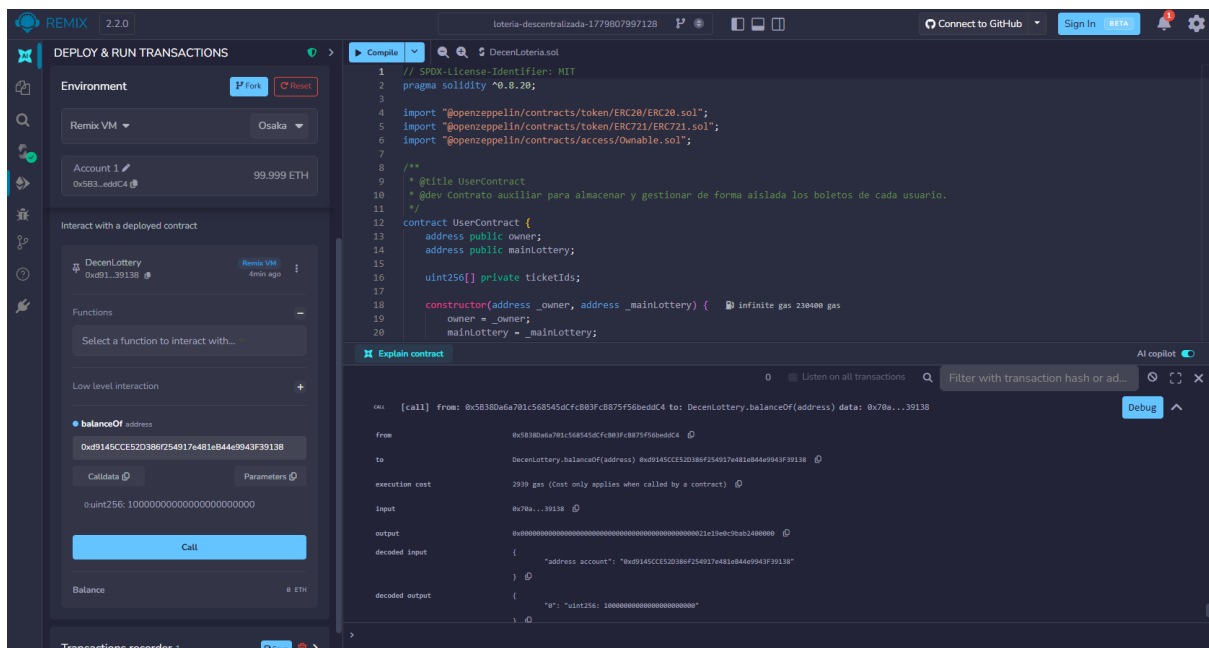


Figura 2: Llamada a balanceOf en la dirección del contrato principal (0xd91...39138) desde la cuenta Account 1, confirmando la correcta reserva de 10,000 tokens.

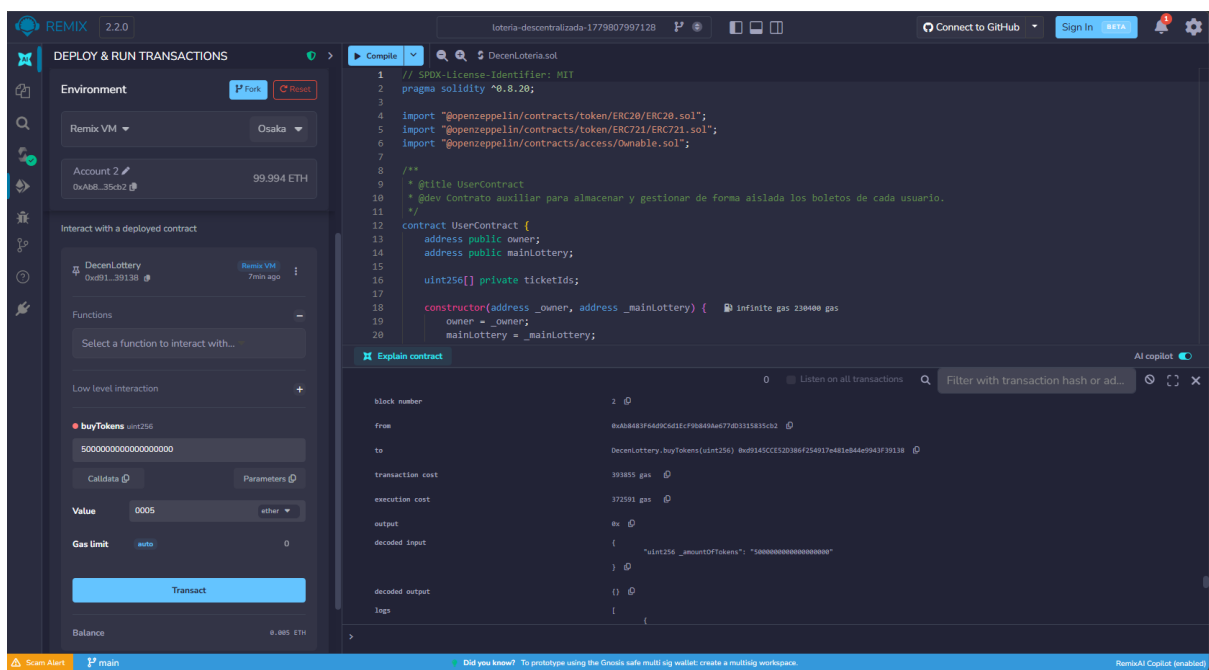


Figura 3: Ejecución exitosa del método buyTokens desde la cuenta Account 2, adquiriendo 5 tokens completos (5000000000000000000) enviando 0.005 ETH de valor.

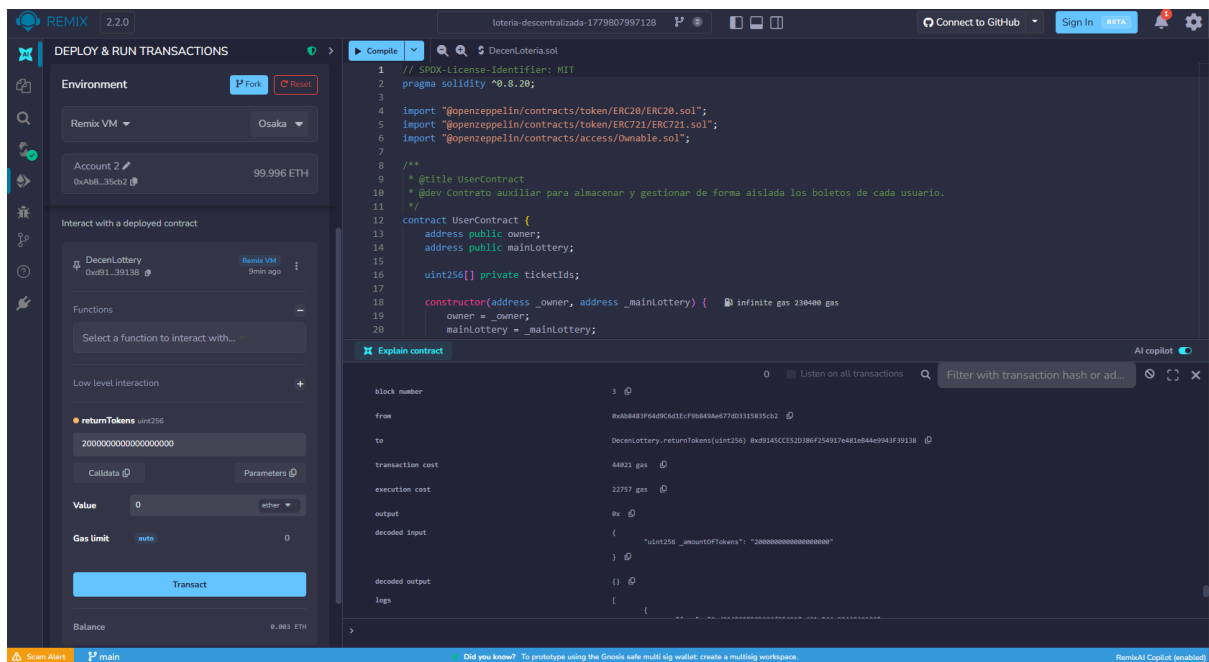


Figura 4: Consulta del saldo del usuario mediante balanceOf para Account 2, retornando un remanente exacto de 3 tokens tras completarse la compra y la posterior devolución.

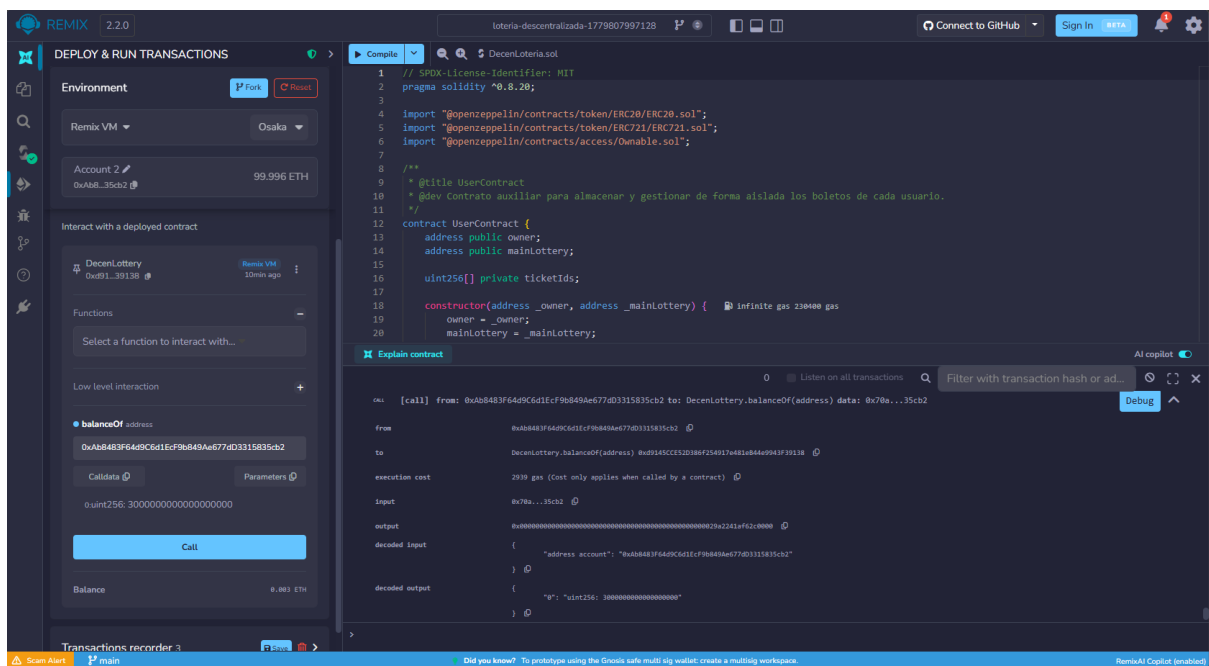


Figura 5: Transacción exitosa de returnTokens iniciada por el usuario (Account 2) para liquidar y devolver 2 tokens de utilidad al contrato principal.

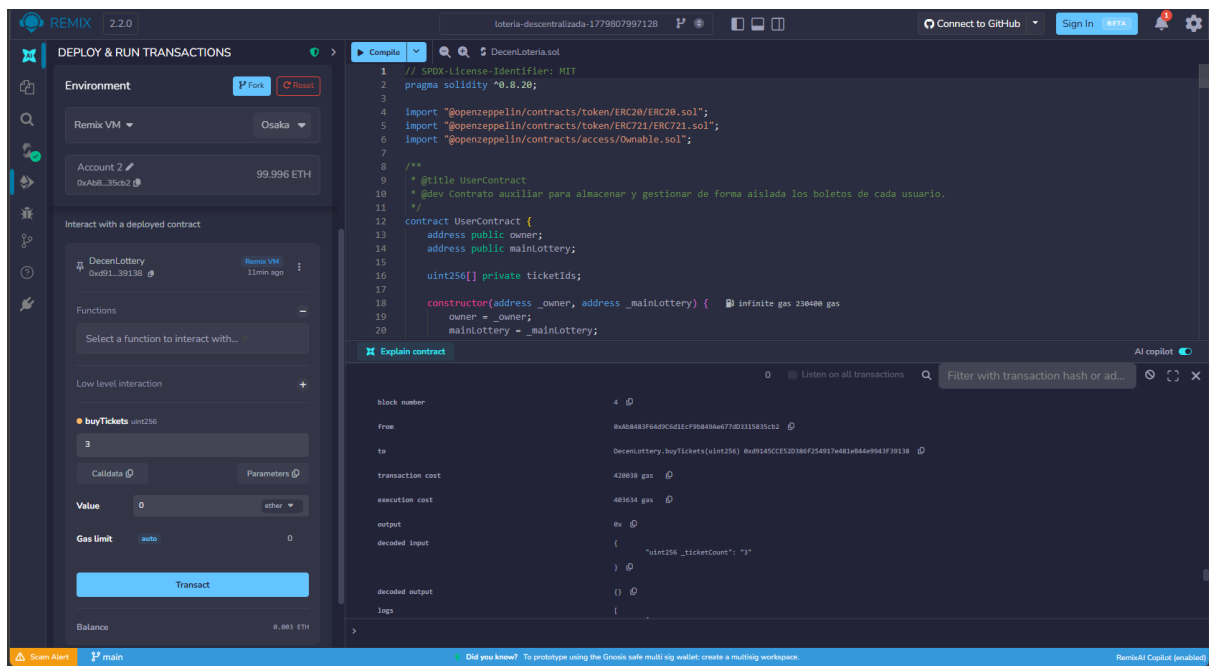


Figura 6: Llamada de transacción a buyTickets para la compra en bloque de 3 boletos de lotería gastando los tokens ERC-20 correspondientes.

4.2. Fase de Compra de Boletos, NFTs y Sorteo

En la segunda fase de pruebas, el usuario interactuó con el mecanismo de la lotería comprando boletos utilizando sus tokens ERC-20. Tras la compra, se comprobó que el balance de tokens del usuario disminuyera de forma proporcional y se consultó la función de lectura de boletos, retornando con éxito la lista de identificadores adquiridos.

Para validar las restricciones de seguridad del estándar ERC-721, se intentó acuñar un boleto NFT llamando directamente al contrato de NFTs desde una cuenta sin autorización, comprobando que la transacción revertía tal como se esperaba. Finalmente, el administrador inició el sorteo del ganador; el contrato seleccionó un boleto al azar, identificó al comprador y liquidó el balance de Ether acumulado, distribuyendo la comisión del 10 % para el propietario y el premio restante del 90 % para la cuenta del ganador seleccionado.

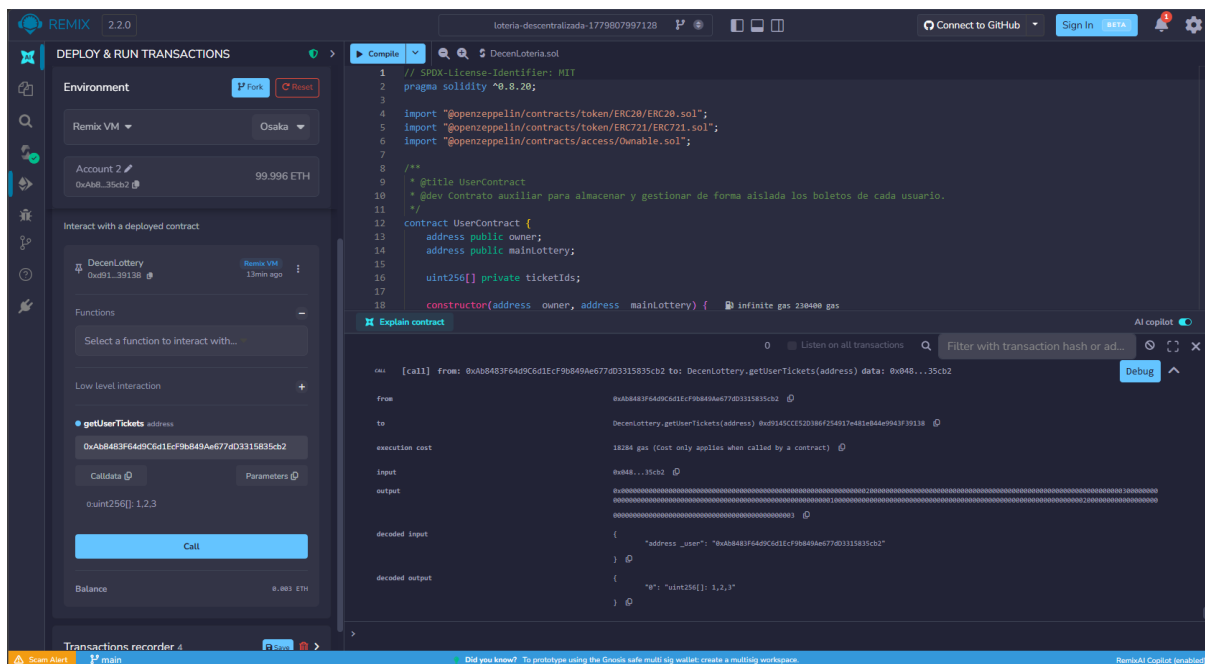


Figura 7: Consulta del método `getUserTickets` para la billetera de Account 2, retornando con éxito la lista de sus 3 boletos guardados en su contrato auxiliar: [1, 2, 3].

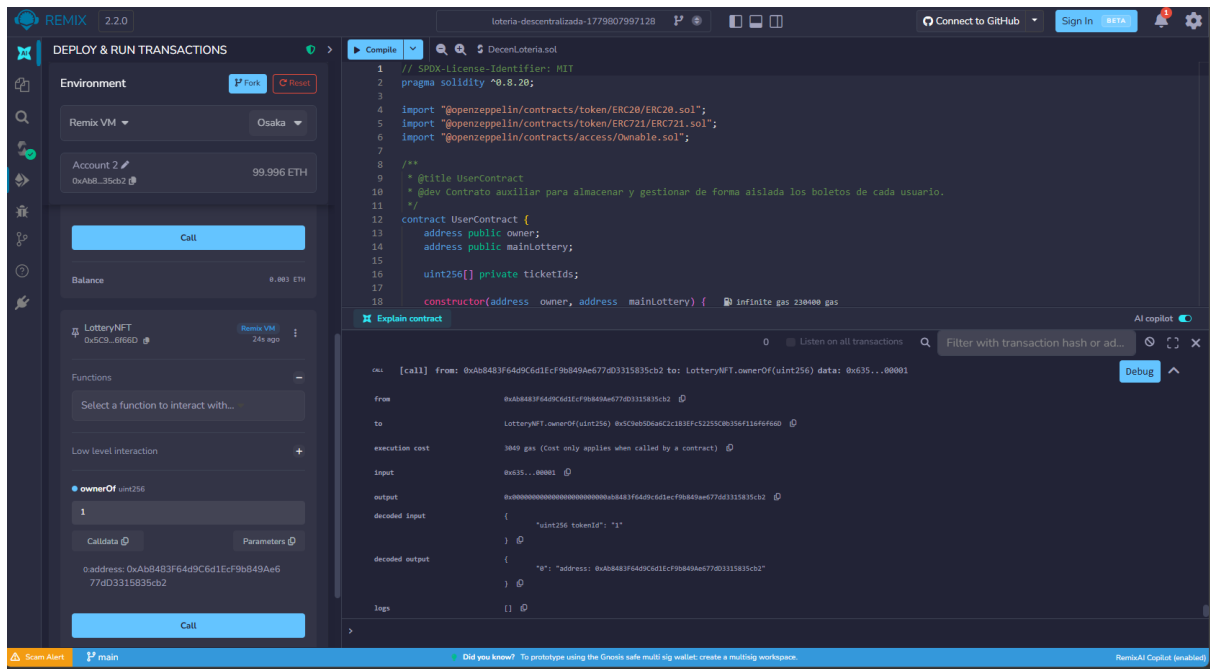


Figura 8: Llamada de lectura de ownerOf en el contrato LotteryNFT cargado dinámicamente, verificando que el boleto ID 1 pertenece a la cuenta de pruebas Account 2.

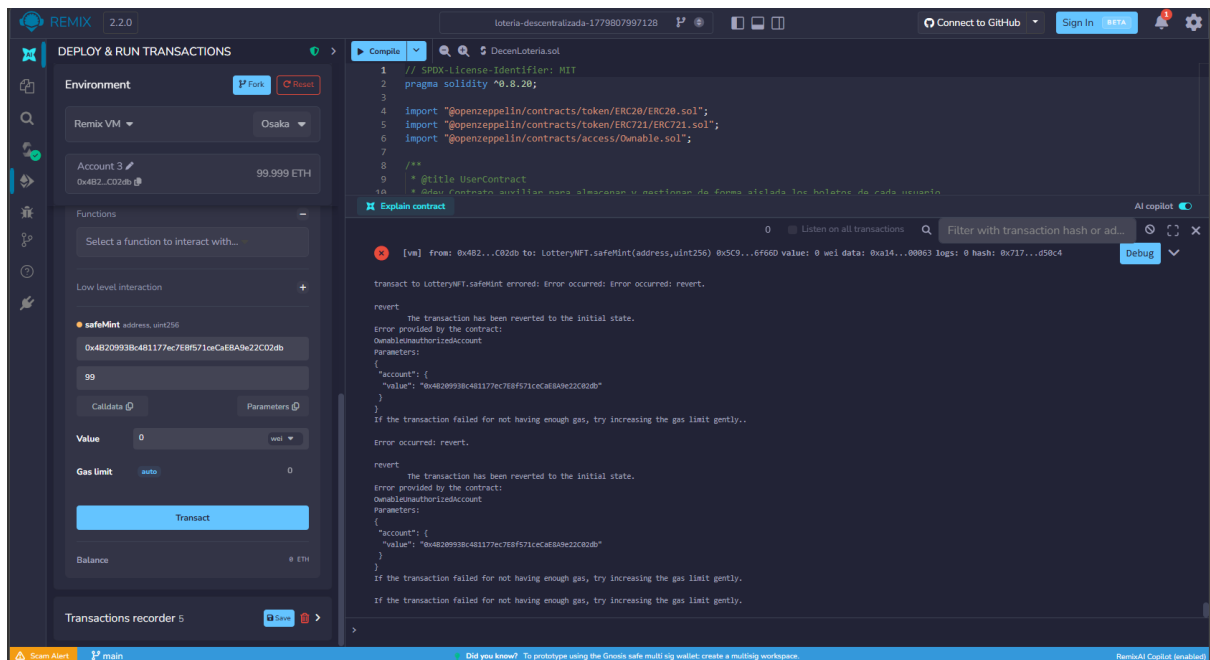


Figura 9: Error de excepción controlado (OwnableUnauthorizedAccount) emitido al intentar acuñar un NFT directamente desde la cuenta Account 3 sin privilegios de Owner.

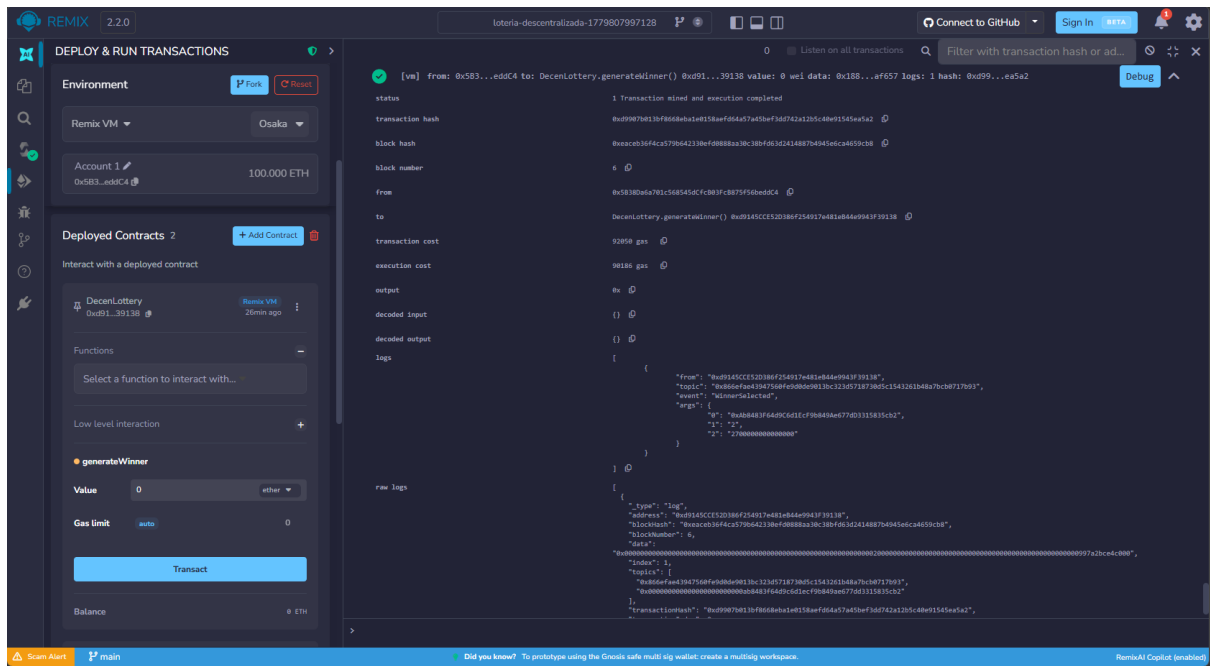


Figura 10: Ejecución del sorteo final generateWinner desde la cuenta administradora Account 1, visualizando el evento WinnerSelected con el boleto número 2 como ganador y un premio neto de 0.0027 ETH.

5. Enlace al Repositorio de GitHub

El código fuente completo de la lotería descentralizada, las versiones del archivo único de Solidity y el registro del desarrollo se encuentran publicados en el siguiente repositorio de GitHub:

`https://github.com/MiltonCuervo/loteria-descentralizada`